

1.1 назначение программы

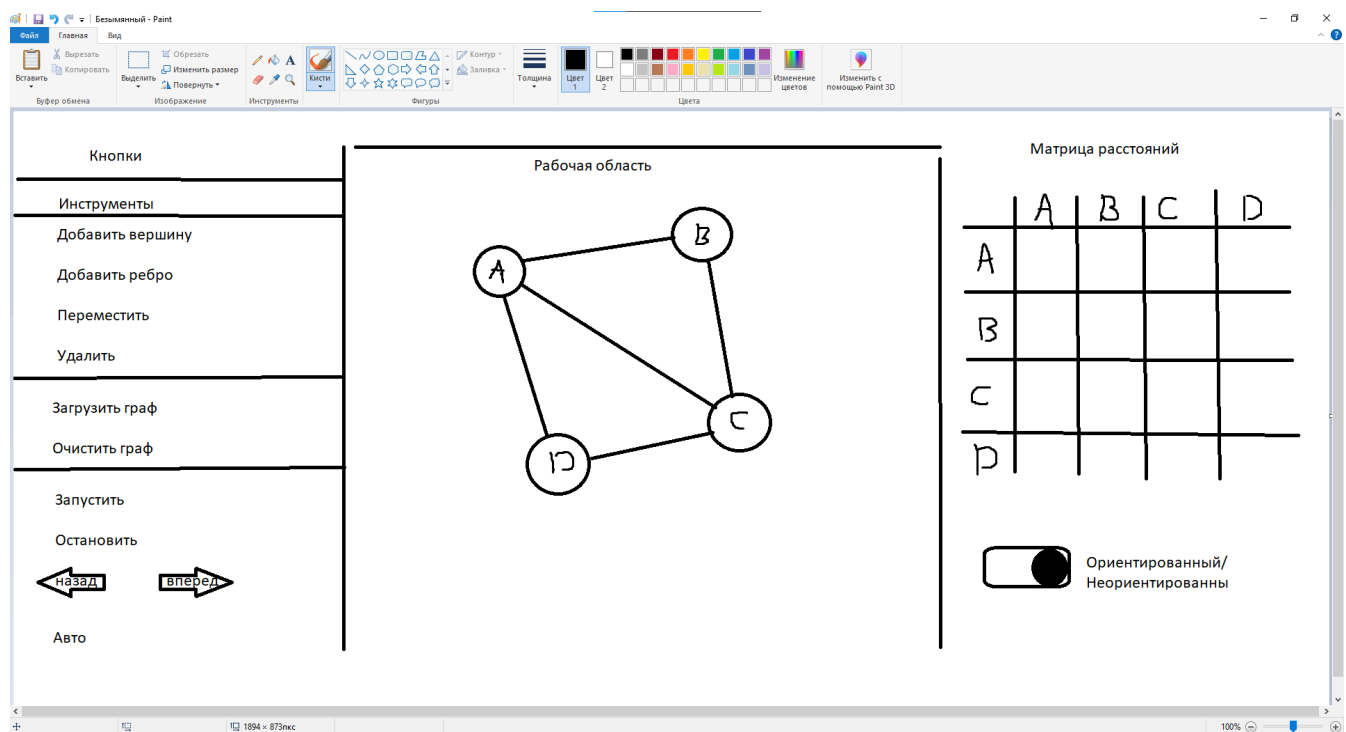
Программа предназначена для визуального изучения алгоритма Флойда-Уоршелла поиска кратчайших путей между всеми парами вершин в взвешенном ориентированном графе.

1.2 Технологический стек

Язык: Kotlin

Kotlin - UI-фреймворк: Compose Multiplatform

2. пользовательский интерфейс



Приложение делится на 4 области. Кнопки, Рабочая область(граф), Матрица и логгер.

2.1. Кнопка «Добавить вершину» - При нажатии кнопки активирует режим создания новой вершины. Клик левой кнопкой мыши (ЛКМ) по свободной области рабочего пространства фиксирует координаты и создает новую вершину графа. Повторное нажатие на кнопку или клавишу Esc отменяет режим.

2.2. Кнопка «Добавить ребро» - При нажатии кнопки активирует режим соединения вершин. Пользователь должен последовательно кликнуть по двум существующим

вершинам: первая станет началом ребра, вторая - концом. После выбора второй вершины на экране появляется окно для ввода числового значения веса ребра.

2.3. Кнопка «Переместить» - При нажатии кнопки позволяет захватить любую вершину (зажатием ЛКМ) и изменить ее координаты на рабочей области. При перетаскивании все инцидентные (связанные с ней) ребра динамически перерисовываются, следуя за вершиной.

2.4. Кнопка «Удалить» - При нажатии кнопки активирует режим удаления элементов. Клик ЛКМ по вершине удаляет ее вместе со всеми связанными с ней ребрами. Клик по ребру удаляет только выбранное ребро.

2.5. Кнопка «Загрузить граф» - При нажатии кнопки инициирует процесс импорта данных. Открывает стандартное системное окно выбора файла. Система ожидает файл в формате .json, считывает его, проводит валидацию (проверку на корректность структуры) данных, формирует модель графа и отрисовывает его на рабочей области.

2.6. Кнопка «Очистить» - При нажатии кнопки полностью очищает рабочую область от всех вершин и ребер, а также сбрасывает внутреннее состояние программы и историю вычислений.

2.7. Кнопка «Запустить» - Инициирует выполнение алгоритма на текущем графе. По умолчанию переводит визуализатор в режим пошагового выполнения: при нажатии происходит расчет и отрисовка первого шага алгоритма с выделением активных элементов.

2.8. Кнопка «Остановить» - При нажатии кнопки приостанавливает выполнение алгоритма. Текущее состояние графа и промежуточные вычисления сохраняются, что позволяет пользователю проанализировать результат.

2.9. Кнопки «Назад» и «Вперед» - Инструменты навигации по истории состояний графа в процессе выполнения алгоритма. Кнопка «Назад» откатывает визуализацию на один шаг назад, а «Вперед» - продвигает на один шаг вперед. Необходимы для детального, кадрового изучения логики работы алгоритма.

2.10. Кнопка «Авто» - При нажатии кнопки запускается непрерывная анимация выполнения алгоритма от начального состояния до завершения.

2.11. Переключатель «Ориентированный / Неориентированный» - Определяет тип создаваемого и загружаемого графа. В неориентированном графе ребра не имеют направления (отображаются как простые линии), а связь между вершинами является двусторонней. В ориентированном графе ребра имеют направление (отображаются со стрелками), что влияет на визуальное представление и логику обхода графа алгоритмами.

3. Описание работы логгера

3.1. Назначение модуля Журнал хода алгоритма (логгер) предназначен для текстового сопровождения работы алгоритма Флойда–Уоршелла в режиме реального времени. Поскольку данный алгоритм выполняет многократные проверки всех пар вершин через промежуточные узлы, визуальное представление изменений на графе недостаточно для полного понимания логики работы.

3.2. Принцип формирования сообщений Для алгоритма Флойда–Уоршелла характерно большое количество внутренних проверок: при n вершинах выполняется n^3 операций сравнения. Для каждого этапа k (промежуточной вершины) выводится заголовок, обозначающий начало проверки; отдельные сообщения формируются только в случае успешного улучшения расстояния между какой-либо парой вершин; если на текущем этапе улучшений не произошло, выводится краткое уведомление об этом. Такой подход позволяет пользователю сосредоточиться на содержательных изменениях, не отвлекаясь на рутинные проверки, не приведшие к обновлению таблицы расстояний.

3.3. Структура записи в журнале

Каждая запись логгера содержит следующую информацию:

- номер этапа (порядковый номер промежуточной вершины k);
- имя промежуточной вершины (вершина, через которую производится проверка)
- текстовое описание действия
- числовые значения - предыдущее и новое расстояние, а также компоненты нового пути (расстояние от i до k и от k до j).

3.4. Примеры сообщений

Ниже приведены типовые формулировки, генерируемые логгером в различных ситуациях:

1. Начало этапа: “[Этап 1] Проверяем пути через вершину A”.
2. Успешное улучшение маршрута: “Улучшен маршрут $B \rightarrow D$: было 12, стало 7 (путь $B \rightarrow A \rightarrow D$: $3 + 4$)”.
3. Отсутствие улучшений на этапе: “На данном этапе улучшений маршрутов не обнаружено”.

4. Невозможность построения пути: “Путь $C \rightarrow E$ через вершину A невозможен: отсутствует ребро $A \rightarrow E$ ”.
5. Завершение алгоритма: “ [Завершение] Алгоритм выполнен за 4 этапа. Найдены кратчайшие расстояния между всеми парами вершин. Всего улучшено 7 маршрутов”.

3.5. Обработка особых случаев

Логгер отдельно реагирует на ситуации, требующие внимания пользователя: 1)

Несвязный граф. Если после завершения алгоритма между некоторыми парами вершин расстояние остаётся равным бесконечности, в итоговом сообщении выводится уведомление:

“Примечание: вершины F и G находятся в разных компонентах связности, путь между ними не существует.”

2) Отрицательные циклы. Алгоритм Флойда–Уоршелла позволяет обнаруживать циклы отрицательного веса: если после завершения на главной диагонали матрицы расстояний появляется отрицательное значение ($d[i][i] < 0$), логгер выводит предупреждение:

“ Вниманию: обнаружен цикл отрицательного веса. Результаты алгоритма могут быть некорректны.”

3.6. Взаимодействие с визуализацией

Логгер работает в синхронизации с графическим представлением графа: при переходе к следующему шагу в журнале появляется новая запись, одновременно на графе выделяются текущая промежуточная вершина k и анализируемая пара (i, j) ; при использовании кнопок «Назад» и «Вперёд» журнал автоматически показывает запись, соответствующую текущему кадру визуализации, что позволяет пользователю сопоставлять текст и графическое состояние. В автоматическом режиме журнал прокручивается синхронно анимацией, формируя описание о ходе работы алгоритма.

4. Визуализация работы алгоритма Флойда–Уоршелла

4.1. Визуализация матрицы расстояний

Интерфейс матрицы спроектирован таким образом, чтобы пользователь мог в любой момент времени однозначно определить, какие ячейки участвуют в текущей операции и как изменяется их содержимое.

4.1.1. Структура и начальное заполнение

Матрица отображается в виде квадратной таблицы размером N^3 , где N - количество вершин в графе. Изначально матрица уже инициализирована и содержит: Главную диагональ - заполненную нулями и окрашенную в нейтральный серый цвет. Ячейки с существующими рёбрами содержат веса соответствующих рёбер графа. Ячейки без прямых соединений - заполненные символом бесконечности (∞), выведенным приглушенным светло-серым цветом.

4.1.2. Цветовое кодирование ячеек

На каждом шаге алгоритма матрица динамически подсвечивается, отражая текущую итерацию трёх вложенных циклов:

Строка i (вершина отправления) - подсвечивается синим фоном.

Столбец j (вершина назначения) - подсвечивается синим фоном.

Ячейка $[i][j]$ (текущая анализируемая) - заключается в жирную контрастную рамку, что делает её главным визуальным акцентом.

Ячейки-слагаемые $[i][k]$ и $[k][j]$ - закрашиваются ярким оранжевым цветом, соответствующим цвету промежуточной вершины k . Такое кодирование обеспечивает наглядное правило: значение в отведенной ячейке сравнивается с суммой двух оранжевых ячеек.

4.1.3. Анимация операций релаксации

Изменения в матрице сопровождаются анимацией:

Обнаружение нового пути (замена ∞ на конечное число): символ бесконечности плавно исчезает, на его месте появляется вычисленное значение. Ячейка кратковременно вспыхивает зелёным цветом, сигнализируя об успехе.

Оптимизация существующего пути (замена большего значения на меньшее): старое число анимировано зачеркивается и заменяется новым. Ячейка также подсвечивается зелёным.

Отклонение неоптимального пути (новое значение больше текущего): старое значение остаётся неизменным, ячейка кратковременно мигает красным цветом.

4.1.4. Итоговое состояние

При завершении алгоритма матрица содержит кратчайшие расстояния между всеми парами вершин. Недостижимые пары по-прежнему отмечены символом ∞ .

4.2. Визуализация графа

Граф является геометрическим представлением задачи, и его визуализация решает две задачи: отображает исходные данные (вершины и рёбра с весами) и синхронно с матрицей показывает, какие элементы участвуют в текущем шаге алгоритма.

4.2.1. Граф отображается в отдельной рабочей области, расположенной рядом с матрицей. Вершины представлены в виде круглых узлов с подписями (именами), ребра - линиями, соединяющими вершины. На каждом ребре размещена метка с числовым значением веса, что позволяет пользователю видеть исходные данные без необходимости обращаться к матрице. Для ориентированного графа рёбра отображаются со стрелками, указывающими направление; для неориентированного - простыми линиями без стрелок.

4.2.2. Цветовое кодирование вершин

Ключевым элементом визуализации графа является строгая цветовая схема, синхронизированная с матрицей. На каждом шаге алгоритма выделяются только активные вершины, а все остальные становятся полупрозрачными, чтобы не отвлекать внимание от текущей операции. Особенность визуализации заключается в том, что цветовое кодирование меняется в зависимости от того, какой именно путь рассматривается алгоритмом в данный момент.

Сценарий А: Оценка прямого пути (участвуют 2 вершины)

Когда алгоритм работает с прямым путем между двумя вершинами (например, на этапе первоначального заполнения матрицы или при обращении к прямому ребру), начальная и конечная вершины визуально разделяются: Вершина i (начальная) - подсвечивается первым цветом (синим). Вершина j (конечная) - подсвечивается вторым цветом (фиолетовым).

Сценарий Б: Поиск пути через промежуточную вершину (участвуют 3 вершины)

Когда алгоритм пытается вычислить или улучшить путь, используя третью вершину для обхода, логика подсветки меняется. Точки отправления и назначения объединяются общим цветом, а промежуточная вершина выделяется контрастным: Вершина i (начальная) и Вершина j (конечная) - подсвечиваются одним цветом (синим). Они символизируют общую границу искомого пути. Вершина k (промежуточная) - подсвечивается другим цветом (фиолетовым). Синхронизация с матрицей Цвета вершин на графе в обоих сценариях полностью соответствуют цветам выделяемых строк, столбцов и ячеек в матрице. Смена цветов создает интуитивно понятный визуальный язык: пользователь с первого взгляда на граф

определяет текущую фазу работы алгоритма. Если горят два разных цвета - идет оценка прямого отрезка; если горят две вершины одного цвета, а между ними выделяется вершина другого цвета - алгоритм выполняет процесс релаксации (поиск более выгодного обходного маршрута), при этом пользователь сразу понимает, какие именно ячейки-слагаемые в матрице участвуют в текущем вычислении.

4.2.3. Состояние неактивных элементов

Все вершины, не участвующие в текущем шаге (кроме i , j и k), отображаются полупрозрачными. Рёбра между неактивными вершинами также становятся менее контрастными. Такой приём реализует принцип «фокусировки внимания»: пользователь концентрируется только на тех элементах, которые обрабатываются алгоритмом в данный момент.

4.2.4. Динамическое поведение

При переходе между шагами алгоритма происходит плавная смена цветовых ролей: Вершина, бывшая на предыдущем шаге промежуточной (k), может стать вершиной отправления (i) на следующем. Цвета перетекают между вершинами, создавая визуальную преемственность и облегчая отслеживание логики алгоритма.

4.2.5. Связь с матрицей

Визуализация графа и матрицы работает как единое целое. Когда пользователь видит в матрице, что ячейка $[A][C]$ обновляется на основе ячеек $[A][B]$ и $[B][C]$, он одновременно видит на графе: синюю вершину A (отправление), фиолетовую вершину B (промежуточная), синюю вершину C (назначение). Это двойное кодирование (числовое в матрице + геометрическое в графе) обеспечивает глубокое понимание алгоритма и позволяет пользователю предугадывать следующие шаги, прежде чем они будут выполнены.

5. формат входных и выходных данных.

5.1 Входные данные:

Граф задающийся через json-файл в формате (начальная вершина, конечная, вес ребра) либо с помощью инструментов графического интерфейса.

5.2 Выходные данные:

Построение конечной матрицы расстояний.

А) Написание прототипа

1. Описание моделей данных
 - 1.1 Vertex
 - 1.2 Edge
 - 1.3 Graph
 - 1.4 AlgorithmStep
 - 1.5 AppState
2. Разметка основного окна
 - 2.1 Панель управления
 - 2.2 Граф (рабочая область)
 - 2.3 Матрица
3. UI дизайн каждой секции, кнопок
4. Написать план тестирования
 - 4.1 Ручное тестирование
 - 4.2 Тестирование алгоритма
 - 4.3 Модульное тестирование
 - 4.4 Интеграционное тестирование

Б) 1-ая версия (Базовый рабочий функционал)

1. Написание алгоритма
2. Написание парсера для ввода/вывода
3. Написание модуля отрисовки для матрицы и графа
4. Тестирование алгоритма

В) 2-ая версия (Является расширенной 1-ой версией: наращивание UI-функционала и добавление интерактивности)

1. Описание функционала кнопок (управление процессом)
2. Тестирование кнопок
3. Разработка прорисовки каждого шага алгоритма (анимации, подсветка)
4. Интеграционное тестирование

Распределение ролей

1. Разработчик логики и состояний

Задача: Математика алгоритма и управление потоком данных .

- Алгоритм: Реализация Флойда-Уоршелла с сохранением каждого шага в список состояний.
- Модели данных: Создание базовых data class для описания шага (какие вершины активны, какой сейчас сценарий — 2 или 3 вершины, старое/новое значение ячейки).
- Навигация: Логика переключения шагов (кнопки Вперед/Назад), выдача текущего состояния.

2. Разработчик графа

Задача: Отрисовка вершин и ребер .

- Отрисовка графа на экране (расположение кружочков и линий).
- Цветовая логика: Чтение текущего состояния и раскраска вершин:
 - Сценарий А (прямой путь): вершина i — синяя, j — фиолетовая.
 - Сценарий Б (через промежуточную): i и j — синие, k — фиолетовая.
- Фокус: Настройка полупрозрачности для всех неактивных вершин на текущем шаге.

3. Разработчик матрицы и интерфейса

Задача: Общая верстка окна, таблица и элементы управления.

- Каркас: Верстка главного экрана, размещение холста с графом, матрицы и панели с кнопками (Play, Pause, Next, Prev).
- Матрица смежности: Отрисовка таблицы $N \times N$, синхронизация цветов строк/столбцов с цветами графа.
- Анимации: Анимация ячеек матрицы (зачеркивание/исчезновение символа ∞ , мигание при успешном обновлении пути).